

Cours détaillé sur SQLAlchemy

1. Introduction à SQLAlchemy

1.1 Qu'est-ce que SQLAlchemy ?

SQLAlchemy est une bibliothèque Python qui permet d'interagir avec des bases de données relationnelles de manière **abstraite, portable et sécurisée**. Elle évite d'écrire du SQL brut en permanence et facilite le passage d'un SGBD à un autre (SQLite, PostgreSQL, MySQL, etc.).

1.2 Pourquoi utiliser SQLAlchemy ?

- Indépendance vis-à-vis du SGBD
- Sécurité (prévention des injections SQL)
- Productivité et maintenabilité du code
- Intégration parfaite avec FastAPI, Flask, Django (hors ORM natif)

1.3 Les deux couches de SQLAlchemy

1. **Core** : manipulation de SQL (tables, requêtes) de manière déclarative
2. **ORM (Object Relational Mapper)** : manipulation des données via des classes Python

👉 Dans ce cours, nous utiliserons **SQLAlchemy ORM**, le plus utilisé en pratique.

2. Installation et prérequis

```
pip install sqlalchemy psycopg2-binary
```

- `sqlalchemy` : bibliothèque principale
- `psycopg2-binary` : driver PostgreSQL
- SQLite est inclus par défaut avec Python

3. Concepts fondamentaux

3.1 Engine

L'**Engine** représente la connexion à la base de données.

```
from sqlalchemy import create_engine  
engine = create_engine("sqlite:///example.db", echo=True)
```

- `echo=True` affiche les requêtes SQL générées

3.2 Session

La **Session** est l'interface entre votre application et la base de données.

```
from sqlalchemy.orm import sessionmaker
SessionLocal = sessionmaker(bind=engine)
session = SessionLocal()
```

3.3 Base déclarative

```
from sqlalchemy.orm import declarative_base
Base = declarative_base()
```

Toutes les classes ORM hériteront de `Base`.

4. Définition d'un modèle (ORM)

Exemple : table `users`

```
from sqlalchemy import Column, Integer, String

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False)
    email = Column(String(150), unique=True, nullable=False)
```

Correspondance ORM : - Classe Python → Table SQL - Attribut → Colonne - Objet → Ligne

5. Création des tables

```
Base.metadata.create_all(bind=engine)
```

Cette commande crée les tables si elles n'existent pas.

PARTIE A : CRUD COMPLET AVEC SQLITE

6. Connexion SQLite

```
DATABASE_URL = "sqlite:///users.db"
engine = create_engine(DATABASE_URL, echo=True)
SessionLocal = sessionmaker(bind=engine)
```

7. CREATE (Insertion)

```
session = SessionLocal()

new_user = User(name="Cheikh", email="cheikh@gmail.com")
session.add(new_user)
session.commit()
session.refresh(new_user)

print(new_user.id)
```

Explications : - `add()` : ajoute à la session - `commit()` : valide dans la base - `refresh()` : récupère les valeurs générées (id)

8. READ (Lecture)

Lire tous les utilisateurs

```
users = session.query(User).all()
```

Lire un utilisateur par ID

```
user = session.query(User).filter(User.id == 1).first()
```

Lire par email

```
user = session.query(User).filter_by(email="cheikh@gmail.com").first()
```

9. UPDATE (Mise à jour)

```
user = session.query(User).filter(User.id == 1).first()
user.name = "Cheikh Ahmad"
session.commit()
```

10. DELETE (Suppression)

```
user = session.query(User).filter(User.id == 1).first()
session.delete(user)
session.commit()
```

PARTIE B : CRUD COMPLET AVEC POSTGRESQL

11. Configuration PostgreSQL

URL de connexion

```
DATABASE_URL = "postgresql+psycopg2://username:password@localhost:5432/
mydatabase"
engine = create_engine(DATABASE_URL, echo=True)
SessionLocal = sessionmaker(bind=engine)
```

👉 Remplacer : - - -

12. Modèle ORM (identique à SQLite)

```
class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False)
    email = Column(String(150), unique=True, nullable=False)
```

👉 Aucun changement du modèle ORM

13. CREATE avec PostgreSQL

```
session = SessionLocal()
user = User(name="Aminata", email="aminata@gmail.com")
session.add(user)
session.commit()
```

14. READ avec PostgreSQL

```
users = session.query(User).all()

user = session.query(User).filter(User.email == "aminata@gmail.com").first()
```

15. UPDATE avec PostgreSQL

```
user = session.query(User).filter_by(email="aminata@gmail.com").first()
user.name = "Aminata Diallo"
session.commit()
```

16. DELETE avec PostgreSQL

```
session.delete(user)
session.commit()
```

17. Bonnes pratiques

- Toujours fermer la session :

```
session.close()
```

- Gérer les exceptions (try / except)

- Séparer :

- models.py
- database.py
- crud.py

18. Comparaison SQLite vs PostgreSQL

Critère	SQLite	PostgreSQL
Usage	Local, tests	Production
Performance	Moyenne	Élevée
Multi-utilisateurs	Limité	Excellent
Configuration	Très simple	Plus complexe

19. Avis pédagogique

👉 **SQLite** est idéal pour l'apprentissage, les TP et les prototypes. 👉 **PostgreSQL** est recommandé pour les projets réels, API FastAPI, systèmes multi-utilisateurs.

Pour un étudiant en **Master IA / DATA**, SQLAlchemy est un **standard incontournable**, notamment avec FastAPI et les architectures modernes.

20. SQLAlchemy avec FastAPI (Architecture API moderne)

20.1 Pourquoi FastAPI + SQLAlchemy ?

- API rapide et moderne
- Validation automatique avec Pydantic
- Documentation Swagger intégrée
- Standard professionnel (startup, recherche, production)

20.2 Architecture professionnelle recommandée

```
project/
├── app/
│   ├── main.py
│   ├── database.py
│   ├── models/
│   │   └── user.py
│   ├── schemas/
│   │   └── user.py
│   ├── crud/
│   │   └── user.py
│   └── routers/
│       └── user.py
└── requirements.txt
```

20.3 Connexion base de données (database.py)

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

DATABASE_URL = "sqlite:///./app.db"
engine = create_engine(DATABASE_URL, connect_args={"check_same_thread":
False})
SessionLocal = sessionmaker(bind=engine)
Base = declarative_base()
```

20.4 Modèle ORM (models/user.py)

```
from sqlalchemy import Column, Integer, String
from app.database import Base

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, nullable=False)
    email = Column(String, unique=True, nullable=False)
```

20.5 Schéma Pydantic (schemas/user.py)

```
from pydantic import BaseModel

class UserCreate(BaseModel):
    name: str
    email: str

class UserRead(UserCreate):
    id: int

class Config:
    from_attributes = True
```

20.6 CRUD (crud/user.py)

```
from sqlalchemy.orm import Session
from app.models.user import User

def create_user(db: Session, user):
```

```
db_user = User(**user.dict())
db.add(db_user)
db.commit()
db.refresh(db_user)
return db_user

def get_users(db: Session):
    return db.query(User).all()
```

20.7 Routes FastAPI (routers/user.py)

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session
from app.database import SessionLocal
from app.schemas.user import UserCreate, UserRead
from app.crud.user import create_user, get_users

router = APIRouter(prefix="/users")

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@router.post("/", response_model=UserRead)
def create(user: UserCreate, db: Session = Depends(get_db)):
    return create_user(db, user)

@router.get("/", response_model=list[UserRead])
def read(db: Session = Depends(get_db)):
    return get_users(db)
```

20.8 Main FastAPI (main.py)

```
from fastapi import FastAPI
from app.database import Base, engine
from app.routers import user

Base.metadata.create_all(bind=engine)
app = FastAPI()
app.include_router(user.router)
```

21. Relations SQLAlchemy

21.1 Relation One-To-Many (User → Posts)

```
class Post(Base):
    __tablename__ = "posts"
    id = Column(Integer, primary_key=True)
    title = Column(String)
    user_id = Column(Integer, ForeignKey("users.id"))

    user = relationship("User", back_populates="posts")

User.posts = relationship("Post", back_populates="user")
```

21.2 Relation Many-To-Many (Students ↔ Courses)

```
association = Table(
    "student_course", Base.metadata,
    Column("student_id", ForeignKey("students.id")),
    Column("course_id", ForeignKey("courses.id"))
)
```

22. Exercices corrigés

Exercice 1

Créer une API FastAPI permettant de gérer des étudiants.

👉 Correction : CRUD + modèle Student + routes

Exercice 2

Créer une relation 1-N entre Professeur et Cours.

👉 Correction : utilisation de ForeignKey + relationship

Exercice 3

Créer une relation N-N entre Étudiants et Modules.

👉 Correction : table d'association

23. Avis final

Ce support correspond à un **niveau Licence 3 – Master IA / DATA** et respecte les **standards professionnels modernes** (FastAPI, SQLAlchemy, PostgreSQL). Idéal pour l'enseignement, les projets et la recherche.